自作言語で作った自作 CPU の上で自作 Ruby が動かない

全部自分のコードのとき、どこを疑うか

284km ・ YAPC::Tokyo 2026

今日の話

・作ったものの紹介ではありません

・疑える層がいくつもあるとき、疑う順番をどう決めるか

・推測を、検査に変えるやり方

コンパイラ・CPU・言語処理系の知識は要りません

言えること・言えないこと

言えること	言えないこと
この方法で実際に何が出たか	他のやり方より優れていること
検査が仮説を 2 回否定したこと	デバッグ一般の正しいやり方であること

事例は、この 3 層の 1 つだけです

3 つの層、全部自分のコード

・上: 自作 Ruby 処理系（4 万行超）

・中: 自作コンパイラが出した RISC-V の機械語

・下: 自作 CPU エミュレータ

Ruby 処理系も CPU エミュレータも、自作言語 Mere で書いた

Mere = OCaml などと同じ ML 系の、小さな自作言語

ある日、動かなくなった

CPU を 32 ビットから 64 ビットに広げたら、

```
puts 1 + 1
```

すら通らない

64 ビット化で書き換えたのは、機械語を出す側と動かす側の両方。149 箇所、

1 コミットではない

定石: 下の層から疑わない

・フレームワークより先に、アプリを疑う

・データベースより先に、クエリを疑う

・それでも切り分けが要るなら、層の境界をテストする

境界のテストは、全部通っていた

層	何と比べたか	結果
Ruby 処理系	本家 ruby 3.2.2、162 本	バイト単位で一致
コンパイラの機械語	同じプログラムを C に変換して実行	64 / 32 ビットとも不一致 0
CPU エミュレータ	同じ機械語を QEMU で実行	7 / 7 一致

2026 年 9 月初めの時点

それでも、3 つとも残る

境界のテストが言えるのは「試したプログラムでは一致する」まで

「その層が正しい」とは言っていない

・自作 Ruby 処理系か

・自作コンパイラが出した機械語か

・自作 CPU エミュレータか

見当 1: 直前に手を入れた場所

・64 ビット化で、関数ごとの「変数の置き場」の大きさを書き換えた

・壊れた値の置き場所を追うと、関数が確保した範囲の外だった

・→ 置き場の数を数え間違えて、隣を踏んでいるのでは

見当 2: 前に一度壊れた仕組み

・関数を呼ぶ前に使っている値を退避し、戻ったら書き戻す

・この作業の途中で、一度ここが壊れていた

・呼び出しの前後で、値が変わって見えた

・→ 呼ばれた側が、書き戻し損ねているのでは

どちらも、証拠つき

誰でもまずここを疑う

そして、2 回とも外れた

推測を、検査に変える

・コンパイラは関数ごとに「変数を何個ぶん置けるか」を先に決めて確保する

・確保した数と、実際に使った数を、コンパイラ自身に比べさせる

・合わなければ、コンパイルを止める

機械が言った

reserved 2, used 17

関数 gc_collect で、確保 2 個・使用 17 個

疑いは当たっていた

直しても、消えない

・溢れは、どのプログラムでもゼロになった

・それでも puts 1 + 1 は通らない

・実在の欠陥だった。でも、原因ではなかった

・推測のままなら「直し方が悪かったのかも」と迷い続けたはず

見当 2 も、検査に

- CPU エミュレータに、関数呼び出しの約束を見張らせる
- 呼ぶ前の値を覚えておき、戻ったときに照合する
- 結果: 一度も鳴らない
- → 書き戻し損ねている関数は、ない

検査は、外れも確定させる

当たれば、そこを直す

外れれば、その分岐が閉じる

どちらでも、次に進める

観察が、食い違い始めた

・実行中の値を出力するコードを足すほど、結果が互いに矛盾した

・ある監視は「書き換えなし」、別の監視は「書き換えあり」

・そこから出した結論:「壊れた値は、メモリのどこかにある」

推理をやめて、出来上がった機械語を読む

逆アセンブル = 機械語を、人が読める命令の列に戻すこと

壊れていた値（Ruby が変数を探すための表）を、読みにいく最初の命令は——

mv a0, s10

3fb164: 000d0513

・メモリからではなく、CPU 自身の置き場（レジスタ s10）から取っている

・「メモリのどこか」は、丸ごと誤りだった

・その関数の中で s10 に書く命令は、4 つだけ

・うち 1 つが、例外から戻るときの書き戻しだった

例外の仕組み

・失敗するかもしれない処理の前に、今の値を退避する

・失敗したら、退避した値を書き戻す

・退避には置き場所が要るので、メモリを一区画確保する

よくある作り

書いていた先は、区画の外

意図	実際	
確保した区画	15 個ぶん（0〜14）	15 個ぶん（0〜14）
値の退避先	5〜14	20〜29

番号は 0 から。区画の外は、まだ誰にも渡していない空き地

空き地は、後から払い出される

1. 区画を確保する
2. 値を、区画の外（空き地）に退避する
3. 処理の途中でメモリが要求され、その空き地が払い出される
4. 退避した値の上に、別のデータが書かれる
5. 失敗して書き戻すと、その別のデータが戻ってくる

壊れていたのは 1 行

前: 20 + i × 4

先頭から 20 バイト目。区画の中で、正しかった

後: (20 + i) × wsz

先頭から 20 個目。wsz は 1 個のバイト数

32 / 64 ビット両対応のための、機械的な置き換え

1 個 = 8 バイトなので、区画のはるか外

最初の見当は、方向としては合っていた

・直前に手を入れた場所 = 64 ビット化の作業

・原因は、同じ作業の別の 1 行だった

・どこをどう測るかを決められなければ、直せないままだった

なぜ、誰も見なかったのか

1. 2 つの検査は、見えない場所にいた

・置き場の数の検査: 関数の区画しか見ない。退避先は別の区画

・呼び出しの約束の検査: 「呼んで、戻る」の対しか見ない。例外の巻き戻しは「戻る」ではない

・どちらも黙っていた。どちらも正直だった

2. テストはあった。動かした軸が 1 本だった

・「例外から戻ると値が戻る」を確かめるテストは、ずっとあった

・退避する値は 8 個。入れ子も、失敗も成功も試していた

・でも、失敗する処理の中でメモリを確保していなかった

・確保 0 回なら通る。1 回で壊れる（バグ入りの版で実測）

3. 自作の層には、借りられるテストがない

・Ruby 処理系は、本家の ruby に答え合わせをさせられる

・自作言語の例外処理を試してくれる人は、どこにもいない

・正解は借りられても、問いは借りられない

・そして原因は、まさにその層にいた

正直な 1 点

・32 ビットも、同じ変更からずっと壊れていた。症状が出なかっただけ

・64 ビットに移った時点で、32 ビットで回すのをやめていた

・壊れたと気づく手段を、自分で捨てていた

・ただし戻れても、窓は 149 箇所。どの 1 行かは別の問題

持ち帰ってほしいこと

自分が正しいと思っていることを、自分以外が判定できる形に置き換える

やること	なぜ効くか
疑いは検査にする	外れも確定する
黙っている検査を信じすぎない	見えない場所がある
観察が食い違ったら、成果物を読む	成果物は構造を述べている

当てるためではなく、外すために効く

いちばん効いたのは、検査が自分の仮説を否定した 2 回だった

自分が書いていない層でも

・「下の層は正しい」は、いつでも仮定

・フレームワークでも、データベースでも、クラウドでも

・仮定が外れたときにやることは、変わらない

ありがとうございました

- Mere: https://merelang.org/
- CPU エミュレータ: https://github.com/284km/memu
- Ruby 処理系: https://github.com/284km/mere-ruby
- この資料: https://284km.github.io/decks/